



Universidad Autónoma del Estado de México

Integrantes de equipo:

Arriaga Cortes Hannia Monserrat

González Vanda Jennifer Aracely

García Velazquez Alejandro

Hernández Rojas Liam Natanael

Asignatura: Paradigmas de programación II

Profesor: Mora Martínez Yeredith Giovana

Fecha de entrega: 05-junio-2026

Tema: Conexión Maestro – Maestro en MariaDB

índice

Introducción.....	3
Objetivo	4
Conexión Maestro – Maestro en MariaDB	5
Descripción de la arquitectura.....	5
Proceso de configuración.....	5
Instalación de MariaDB.....	5
Configuración de my.ini	5
Reiniciar MariaDB	6
Crea usuario de replicación	6
Ver estado del maestro.....	7
Configurar conexión al maestro	7
Iniciar la replica	8
Verificar si funciona	8
Creación de la base de datos	8
Ventajas de la configuración Maestro – Maestro	15
Aplicación en el proyecto	16
Conclusión.....	17

Introducción

La replicación Maestro – Maestro en MariaDB es una técnica que permite que dos servidores de bases de datos funcionan simultáneamente como maestro y esclavo. Esto significa que ambos servidores pueden recibir operaciones de escritura y lectura, mientras intercambian automáticamente los cambios realizados para mantener la información sincronizada.

Esta configuración ofrece alta disponibilidad, redundancia y tolerancia a fallos, ya que, si uno de los servidores deja de funcionar, el otro puede continuar operando sin afectar significativamente el servicio.

Objetivo

Implementar una arquitectura de replicación Maestro – Maestro utilizando MariaDB para generar la sincronización bidireccional de la información almacenada en las bases de datos, mejorando la disponibilidad y confiabilidad del sistema.

Conexión Maestro – Maestro en MariaDB

Descripción de la arquitectura

La arquitectura implementada consta de dos servidores:

- Servidor A (Maestro 1)
- Servidor B (Maestro 2)

Cada servidor actúa simultáneamente como:

- Maestro: envía sus cambios al otro servidor.
- Esclavo: recibe y aplica los cambios provenientes del otro servidor.

De esta manera, cualquier inserción, actualización o eliminación realizada en cualquiera de los dos servidores es replicada automáticamente al servidor opuesto.

Proceso de configuración

Instalación de MariaDB

Se instaló MariaDB en ambos equipos participantes de la replicación. Posteriormente se verificó que el servicio estuviera funcionando correctamente.

Configuración de my.ini

A cada servidor se le asignó un identificador único (server – id) para que MariaDB pudiera distinguir el origen de las transacciones replicadas, queda de la siguiente manera:

- Computadora 1:

log-bin

server_id=1

log-basename=master1

binlog-format=mixed

bind-address = 0.0.0.0

```
[mysqld]  
log-bin  
server_id=1  
log-basename=master1  
binlog-format=mixed  
bind-address = 0.0.0.0
```

- Computadora 2:

log-bin

server_id=2

log-basename=master2

binlog-format=mixed

```
[mysqld]  
log-bin  
server_id=2  
log-basename=master2  
binlog-format=mixed  
bind-address = 0.0.0.0
```

Reiniciar MariaDB

En ambas computadoras, entra en servicios de Windows y busca la aplicación de “Servicios”, buscas la aplicación de MariaDB y procedes de poner clic derecho para seleccionar la opción de “Reiniciar”. Cierras la aplicación de MariaDB y volvemos a abrir.

Crea usuario de replicación

En ambos equipos, realiza la inserción de su usuario para poder continuar con la replicación les asignamos todos los permisos, quedando de la siguiente manera:

- **Equipo/Computadora 1**

```
CREATE USER 'ara'@'%' IDENTIFIED BY '123';
```

```
GRANT REPLICATION SLAVE ON *.* TO 'ara'@'%';
```

FLUSH PRIVILEGES;

- **Equipo/Computadora 2**

```
CREATE USER 'liem@'%' IDENTIFIED BY '123';
```

```
GRANT REPLICATION SLAVE ON *.* TO 'liem'@'%';
```

FLUSH PRIVILEGES;

Se creó un usuario especializado para la replicación con los privilegios necesarios para acceder a los registros binarios y el usuario recibió permisos de tipo REPLICATION SLAVE para permitir la transferencia de información entre ambos servidores.

Ver estado del maestro

En ambos equipos, ejecuta en MariaDB la siguiente línea de código:

```
SHOW MASTER STATUS;
```

File	Position	Binlog_Do_DB	Binlog_Ignore_DB	Gtid_Binlog_Pos
master1-bin.000001	9054			0-1-20

1 row in set (0.000 sec)

Configurar conexión al maestro

Entrar en MariaDB y ejecutar:

- **Equipo/Computadora 1**

```
CHANGE MASTER TO
```

```
MASTER_HOST='172.20.10.5', (anotar la IP de la máquina del maestro)
```

```
MASTER_USER='liem', (nombre del usuario del maestro)
```

```
MASTER_PASSWORD='123', (contraseña del usuario)
```

```
MASTER_PORT=3306, (puerto por el cual se va a conectar)
```

```
MASTER_LOG_FILE='master2-bin.000001', (anotar lo que nos salió en el paso 5)
```

MASTER_LOG_POS=330; (anotar la segunda columna del paso 5)

- **Equipo/Computadora 2**

CHANGE MASTER TO

MASTER_HOST='172.20.10.4', (anotar la IP de la máquina del maestro)

MASTER_USER='ara', (nombre del usuario del maestro)

MASTER_PASSWORD='123', (contraseña del usuario)

MASTER_PORT=3306, (puerto por el cual se va a conectar)

MASTER_LOG_FILE='master1-bin.000001', (anotar lo que nos salió en el paso 5)

MASTER_LOG_POS=331; (anotar la segunda columna del paso 5)

Iniciar la replica

Entrar nuevamente cada equipo en MariaDB y colocar el siguiente código:

START SLAVE;

Verificar si funciona

Si todo salió correctamente, lo podremos verificar con la siguiente línea de código, quedando de la siguiente manera:

SHOW SLAVE STATUS\G

Debe de salir:

Slave_IO_Running: Yes

Slave_SQL_Running: Yes

Si ambos dicen YES significa que la réplica funciona correctamente.

Creación de la base de datos

Copiar y pegar el siguiente código para que funcione el programa de la base de datos:

- Creación de la base de datos y usarla:

Create database aero;

use aero;

- Creación de la tabla de empleado

create table empleado(

idEmpleado bigint primary key,

nombre varchar(45),

apellidoPaterno varchar(45),

apellidoMaterno varchar(45),

fechaNacimiento date,

sexo char,

rfc varchar(45),

telefono varchar(45),

estatus tinyint,

sueldo double,

area varchar(45));

- Creación de la table de vuelo

create table vuelo(

idVuelo bigint primary key,

origen varchar(45),

destino varchar(45),

noAsientos int,

precio double,

fechaInicio date,

fechaLlegada date,

estatus tinyint);

- Creación de la tabla auditoria de vuelo

```
create table audi_vuelo(
```

```
id int primary key auto_increment,
```

```
origenAnt varchar(45),
```

```
destinoAnt varchar(45),
```

```
noAsientosAnt int,
```

```
precioAnt double,
```

```
fechaInicioAnt date,
```

```
fechaLlegadaAnt date,
```

```
estatusAnt tinyint,
```

```
origenNue varchar(45),
```

```
destinoNue varchar(45),
```

```
noAsientosNue int,
```

```
precioNue double,
```

```
fechaInicioNue date,
```

```
fechaLlegadaNue date,
```

```
estatusNue tinyint,
```

```
usuario varchar(50),
```

```
modificado datetime,
```

```
accion varchar(50),
```

idVuelo bigint);

- Creación de la table auditoria de empleado

```
create table audi_empleado(  
    id int primary key auto_increment,  
    nombreAnt varchar(45),  
    apellidoPaternoAnt varchar(45),  
    apellidoMaternoAnt varchar(45),  
    fechaNacimientoAnt date,  
    sexoAnt char,  
    rfcAnt varchar(45),  
    telefonoAnt varchar(45),  
    estatusAnt tinyint,  
    sueldoAnt double,  
    areaAnt varchar(45),  
    nombreNue varchar(45),  
    apellidoPaternoNue varchar(45),  
    apellidoMaternoNue varchar(45),  
    fechaNacimientoNue date,  
    sexoNue char,  
    rfcNue varchar(45),  
    telefonoNue varchar(45),  
    estatusNue tinyint,  
    sueldoNue double,
```

```
areaNue varchar(45),  
usuario varchar(50),  
modificado datetime,  
accion varchar(50),  
idEmpleado bigint);
```

- Creación de triggers para la tabla de Empleado

Actualización

```
create trigger modificaEmpleado  
after update on empleado  
for each row  
insert into audi_empleado(  
nombreAnt, apellidoPaternoAnt, apellidoMaternoAnt, fechaNacimientoAnt, sexoAnt,  
rfcAnt, telefonoAnt, estatusAnt, sueldoAnt, areaAnt,  
nombreNue, apellidoPaternoNue, apellidoMaternoNue, fechaNacimientoNue,  
sexoNue, rfcNue, telefonoNue, estatusNue, sueldoNue, areaNue,  
usuario, modificado, accion, idEmpleado)  
values(  
old.nombre, old.apellidoPaterno, old.apellidoMaterno, old.fechaNacimiento,  
old.sexo, old.rfc, old.telefono, old.estatus, old.sueldo, old.area,  
new.nombre, new.apellidoPaterno, new.apellidoMaterno, new.fechaNacimiento,  
new.sexo, new.rfc, new.telefono, new.estatus, new.sueldo, new.area,  
user(), now(), 'Modificado', new.idEmpleado);
```

Insertión

create trigger agregaEmpleado

after insert on empleado

for each row

insert into audi_empleado(

nombreNue, apellidoPaternoNue, apellidoMaternoNue, fechaNacimientoNue,
sexoNue, rfcNue, telefonoNue, estatusNue, sueldoNue, areaNue,

usuario, modificado, accion, idEmpleado)

values(

new.nombre, new.apellidoPaterno, new.apellidoMaterno, new.fechaNacimiento,
new.sexo, new.rfc, new.telefono, new.estatus, new.sueldo, new.area,

user(), now(), 'Agregado', new.idEmpleado);

Eliminación

create trigger eliminaEmpleado

after delete on empleado

for each row

insert into audi_empleado(

nombreAnt, apellidoPaternoAnt, apellidoMaternoAnt, fechaNacimientoAnt, sexoAnt,
rfcAnt, telefonoAnt, estatusAnt, sueldoAnt, areaAnt,

usuario, modificado, accion, idEmpleado)

values(

old.nombre, old.apellidoPaterno, old.apellidoMaterno, old.fechaNacimiento,
old.sexo, old.rfc, old.telefono, old.estatus, old.sueldo, old.area,

user(), now(), 'Eliminado', old.idEmpleado);

- Creación de triggers para la tabla de Vuelo

Actualización

create trigger modificaVuelo

after update on vuelo

for each row

insert into audi_vuelo(

origenAnt, destinoAnt, noAsientosAnt, precioAnt, fechaInicioAnt, fechaLlegadaAnt, estatusAnt,

origenNue, destinoNue, noAsientosNue, precioNue, fechaInicioNue, fechaLlegadaNue, estatusNue,

usuario, modificado, accion, idVuelo)

values(

old.origen, old.destino, old.noAsientos, old.precio, old.fechaInicio, old.fechaLlegada, old.estatus,

new.origen, new.destino, new.noAsientos, new.precio, new.fechaInicio, new.fechaLlegada, new.estatus,

user(), now(), 'Modificado', new.idVuelo);

Inserción

create trigger agregaVuelo

after insert on vuelo

for each row

insert into audi_vuelo(

origenNue, destinoNue, noAsientosNue, precioNue, fechaInicioNue, fechaLlegadaNue, estatusNue,

usuario, modificado, accion, idVuelo)

values(

new.origen, new.destino, new.noAsientos, new.precio, new.fechaInicio,
new.fechaLlegada, new.estatus,

user(), now(), 'Agregado', new.idVuelo);

Eliminación

create trigger eliminaVuelo

after delete on vuelo

for each row

insert into audi_vuelo(

origenAnt, destinoAnt, noAsientosAnt, precioAnt, fechaInicioAnt, fechaLlegadaAnt,
estatusAnt,

usuario, modificado, accion, idVuelo)

values(

old.origen, old.destino, old.noAsientos, old.precio, old.fechaInicio,
old.fechaLlegada, old.estatus,

user(), now(), 'Eliminado', old.idVuelo);

Ventajas de la configuración Maestro – Maestro

- Alta disponibilidad del sistema.
- Redundancia de la información.
- Mayor tolerancia a fallos.
- Distribución de la carga de trabajo.
- Continuidad operativa ante la caída de un servidor.

Aplicación en el proyecto

La replicación Maestro – Maestro fue implementada en el sistema de gestión de vuelos desarrollado en C#. La base de datos contiene información relacionada con empleados y vuelos. Gracias a esta configuración, cualquier modificación realizada en cualquiera de los servidores se replica automáticamente al otro, garantizando la integridad y disponibilidad de los datos utilizados por la aplicación.

Conclusión

La implementación de la replicación Maestro – Maestro en MariaDB permitió establecer un entorno de bases de datos robusto y confiable para el sistema desarrollado. Mediante la sincronización bidireccional de la información se logró mantener la consistencia de los datos, mejorar la disponibilidad del servicio y proporcionar una solución escalable para futuras ampliaciones del proyecto.